From First Order to Second: A Hessian Screening Rule for the Lasso

NeurIPS 2022 (Lund University): reusing curvature information to screen predictors tighter and warm-start the solver near-exactly, especially under high correlation

The lasso is one of the most reliable workhorses in high-dimensional statistics: add an ℓ1 penalty to a regression loss and the model both fits and selects features at once. The catch is that the right penalty strength λ is never known in advance. In practice you don't fit one lasso model; you fit an entire path of them across many values of λ and pick the best by cross-validation, refitting the same expensive problem again and again. When you have tens of thousands of predictors, that whole path is where the wall-clock time goes.

A standard trick to make each fit cheaper is a screening rule: before the solver runs, cheaply guess which predictors will stay at exactly zero and discard them, so the optimizer only ever sees a small candidate set. Screening rules are what make large-scale lasso practical. But the popular ones share an awkward weakness: they get conservative exactly when predictors are strongly correlated, which is the regime where speed matters most. They keep far too many predictors, and the warm starts they hand the solver are inaccurate, so convergence is slow.

This NeurIPS 2022 paper by Johan Larsson and Jonas Wallin traces both problems to the same root cause, first-order information, and fixes them with the same tool. The Hessian Screening Rule uses second-order (curvature) information to estimate the next step of the path. The authors show that a single Hessian does two jobs at once: it screens predictors far more tightly, and it produces a warm start that is essentially the exact solution whenever the active set doesn't change. The result is the fastest method across nearly every simulated and real benchmark they test.

Paper: https://arxiv.org/abs/2104.13026

Code (C++/R): https://github.com/jolars/HessianScreening

Why fitting a whole path is the real cost

Because λ must be tuned, the unit of work is not a single lasso fit but a sequence of them along a decreasing grid of penalties, each seeded from the previous solution. Sequential screening rules like the strong rule and the working-set strategy exploit this: they use the solution at λ_k to predict which predictors can possibly become active at the next, smaller λ_{k+1}. The paper's first observation is unifying: both rules can be rewritten as estimates of the same quantity, the correlation (the negative gradient) at the next step. That reframing exposes their common flaw: the estimate is purely first-order, so it drifts when predictors are correlated, forcing the solver into extra KKT checks and re-optimization.

One Hessian, two jobs

The method rests on a simple fact: on any interval of λ where the active set of nonzero coefficients does not change, the lasso solution is a linear function of λ. Linearity is exactly what a second-order estimate can exploit. Using the Hessian of the active predictors, the authors write down a sharper, second-order estimate of the correlation at the next penalty value:

$$\hat{c}_H(\lambda_{k+1}) = c(\lambda_k) + (\lambda_{k+1} - \lambda_k) \cdot X^T X_A (X_A^T X_A)^{-1} \operatorname{sign} \beta(\lambda_k)_A$$

To keep this affordable the expensive inner products are restricted to the strong-rule set, with a small fraction of the unit bound added back as a safety margin. The same Hessian inverse then pulls double duty as a warm start for the coefficients themselves, and because the solution is linear in λ on a fixed active set, this warm start is not just close but exact whenever the active set stays constant:

$$\beta(\lambda_{k+1})_A = \beta(\lambda_k)_A + (\lambda_k - \lambda_{k+1}) \cdot H_A^{-1} \operatorname{sign} \beta(\lambda_k)_A$$

Two engineering pieces make this practical at scale: low-rank updates keep the Hessian and its inverse current as predictors enter and leave the active set, and an approximate-homotopy scheme adaptively places the λ grid. Crucially, nothing here is specific to squared error. The rule applies to any twice-differentiable smooth loss, so ℓ1-regularized logistic regression comes along for free.

Job one: much tighter screening

The first payoff is how few predictors the rule keeps. The figure below fits a full path on a design with n = 200 observations and p = 20000 predictors at three correlation levels, and plots how many predictors each rule screens in (keeps as candidates) at every step; the dashed line marks the true active-set floor. The y-axis is on a log scale, so vertical gaps are order-of-magnitude gaps.

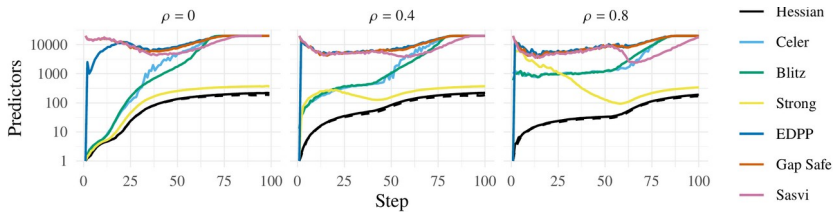

Figure 1. Predictors screened per step at correlations ρ = 0, 0.4, 0.8 (n = 200, p = 20000, log scale, averaged over 20 runs). The Hessian rule (black) tracks the true active-set floor (dashed) closely, while Celer, Blitz, Strong, EDPP, Gap Safe, and Sasvi retain orders of magnitude more predictors, and the gap widens as correlation rises.

The Hessian curve hugs the dashed floor, keeping on the order of a couple hundred predictors, while the safe and heuristic alternatives sit one to three orders of magnitude higher, retaining thousands to tens of thousands. The gap is smallest at ρ = 0 and widens as correlation grows, which is exactly the direction competing rules move the wrong way. Tighter screening means every inner solve the optimizer runs is over a dramatically smaller problem.

Job two: near-exact warm starts

Tighter screening alone doesn't explain the full speedup; the warm start matters just as much. Because the Hessian warm start is the exact solution whenever the active set is unchanged, most path steps should converge almost immediately. The next figure isolates that effect by counting coordinate-descent passes per step for the Hessian warm start versus a standard warm start (the previous step's solution), on two real data sets.

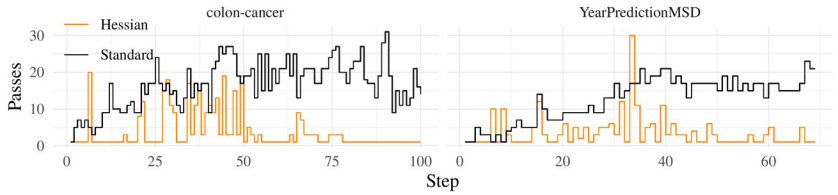

Figure 2. Coordinate-descent passes per path step on colon-cancer (n = 62, p = 2000) and YearPredictionMSD (n = 463715, p = 90), Hessian warm start (orange) versus standard warm start (black). The Hessian warm start collapses most steps to a single pass, because it is near-exact whenever the active set does not change.

The standard warm start needs a fluctuating 10 to 30 passes per step; the Hessian warm start spends most steps at a single pass, spiking only at the steps where the active set actually changes. This is the mechanism behind the method's dominance in the n ⩾ p setting, where the per-pass cost is high and cutting passes pays off directly.

How much faster, end to end

Do tighter screening and better warm starts translate into wall-clock wins? The next figure benchmarks full-path fit time against the working-set strategy, Celer, and Blitz on simulated Gaussian designs, in a low-dimensional regime (n = 10000, p = 100) and a high-dimensional one (n = 400, p = 40000), across least-squares and logistic losses and three correlation levels. Time is relative to the fastest method in each group, so the Hessian bar sits at 1.

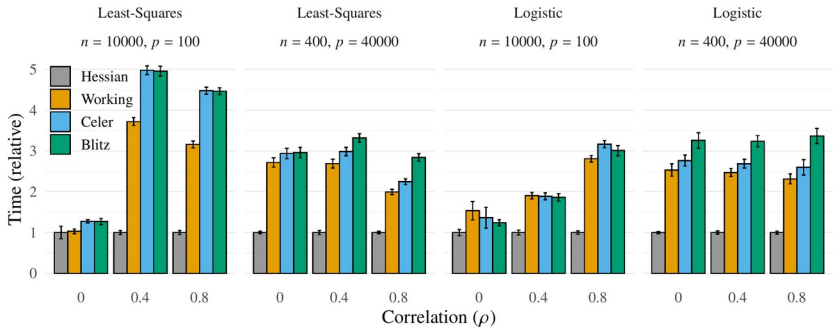

Figure 3. Relative full-path fit time on simulated data (lower is better; Hessian normalized to 1), across least-squares and logistic losses, two dimensional regimes, and correlations ρ = 0, 0.4, 0.8. The Hessian rule is fastest in every configuration; its margin is largest in the high-correlation, low-dimensional least-squares case, where baselines run 3–5× slower.

The Hessian rule is fastest in every single configuration, and its lead is widest precisely where screening rules have historically struggled: the high-correlation, low-dimensional least-squares case, where the working-set strategy, Celer, and Blitz all run roughly three to five times slower. Where correlation is zero and the problem is easy, the margin narrows, an honest

reflection that the gains come from handling the hard regime well rather than from a uniform constant-factor trick.

The same story on real data

The pattern carries over to twelve real data sets spanning gene-expression matrices, text corpora, and problems with millions of features. The table below reports full-path fit time in seconds (lower is better) on a representative subset. On ℓ1-regularized least-squares the Hessian rule is fastest on all five data sets, and in all but one of them it finishes in under half the runner-up's time: 78.8 s versus 541 s on YearPredictionMSD (about 6.9×), and 14.3 s versus 143 s on e2006-tfidf (about 10×).

Table 1. Full-path fit time in seconds on a representative subset of real data sets (lower is better). Hessian is fastest on every least-squares set shown and on the larger logistic sets; the working-set strategy is the usual runner-up. On the full benchmark the lone loss is arcene (logistic), where the working-set strategy is marginally faster.

Data set	n	p	Loss	Hessian	Working	Blitz	Celer
bcTCGA	536	17,322	Least-Squares	3.00	7.67	11.7	10.6
e2006-tfidf	16,087	150,360	Least-Squares	14.3	143	277	335
YearPredictionMSD	463,715	90	Least-Squares	78.8	541	706	712
colon-cancer	62	2,000	Logistic	0.054	0.134	0.177	0.169
madelon	2,000	500	Logistic	48.2	232	240	247
rcv1	20,242	47,236	Logistic	132	266	384	378

On logistic regression the story is nearly as strong, with large wins on madelon (48.2 s versus 232 s) and rcv1 (132 s versus 266 s). Across all twelve sets the single exception is arcene, a small dense logistic problem where the working-set strategy is marginally faster; the authors report that case rather than hide it.

Takeaway, and where it stops

The appeal of the Hessian Screening Rule is that one idea pays off twice. Reusing second-order information tightens screening so the solver sees far fewer predictors, and supplies warm starts so accurate that many steps converge in a single pass. Together they make it the fastest way to fit lasso and ℓ1-logistic regularization paths across the benchmarks here, with the biggest edge in the high-correlation regime that has long been the hardest for screening rules.

The boundaries are stated plainly. Storing the Hessian and its inverse costs more memory than competing methods, which can become prohibitive when min{n, p} is large; there the working-set strategy is the more sensible fallback. And because this is a sequential rule tuned for fitting an entire path, it is not the right tool when you only need the solution at a single λ, where a dynamic method like Celer or Blitz is likely to win. But for the common workflow of fitting a full path and cross-validating on correlated high-dimensional data, and for any composite objective whose smooth part is twice-differentiable, the Hessian Screening Rule is a clear default.